Big Data Mining Algorithms

Module 5

Handling large datasets in main memory

- Handling large datasets in main memory involves techniques and algorithms designed to process and analyze data that may exceed the available RAM. Below are methods and algorithms commonly used for this purpose.

Techniques for Handling Large Datasets in Main Memory

1. **Data Partitioning**:
   1. Split datasets into smaller, manageable chunks that fit into memory.
   2. Each chunk can be processed sequentially or in parallel, reducing memory load.
2. **Out-of-Core Processing**:
   1. Utilize algorithms that can operate on data stored on disk rather than fully loading it into memory.
   2. Libraries like Dask, Apache Spark, and Vaex support out-of-core processing.
3. **Streaming**:
   1. Process data in a continuous stream rather than loading entire datasets.
   2. Useful for real-time analytics and when only a subset of data needs to be analyzed at any moment.
4. **Memory-Mapped Files**:
   1. Use memory-mapped files to allow portions of large files to be accessed as if they were in memory, which helps manage memory usage.
5. **Distributed Computing**:
   1. Use clusters of computers to distribute processing tasks, which can handle larger datasets by utilizing the combined memory of multiple machines.
6. **Data Compression**:
   1. Compress data to fit more information into available memory, often at the cost of increased processing time for decompression.

Algorithms for Handling Large Datasets in Main Memory

1. MapReduce:

1. A programming model used for processing large datasets with a distributed algorithm on a cluster.
2. It splits tasks into smaller sub-tasks (Map phase) and aggregates results (Reduce phase).

2. Streaming Algorithms:

1. Algorithms designed to process data in a single pass (or limited passes), such as:
    1. **Count-Min Sketch**: For estimating frequency of events in data streams.
    2. **Reservoir Sampling**: For selecting a random sample from a stream of data.

3. Batch Processing Algorithms:

1. Algorithms that process data in batches, such as:
    1. **K-Means Clustering**: Can be adapted for large datasets using Mini-Batch K-Means.
    2. **Gradient Descent**: Can be modified to work with mini-batches rather than the entire dataset.

4. Hierarchical Clustering:

1. Algorithms like Agglomerative Clustering can be applied to subsets of data iteratively, merging clusters as more data is processed.

5. Indexing Structures:

1. Data structures like B-trees, R-trees, and hash tables are used for efficient data retrieval, allowing large datasets to be searched more quickly.

6. In-Memory Databases:

1. Databases like Redis or Memcached provide efficient storage and retrieval of large datasets in memory.

7. Approximation Algorithms:

1. Techniques that provide approximate solutions for problems like finding frequent items or heavy hitters without needing to store the entire dataset in memory.

Frequent Pattern Mining

Frequent Pattern Mining

- Frequent pattern mining is the process of finding recurring patterns, such as sets of items, subsequences, or substructures, that appear frequently in a dataset.
- The main goal is to discover patterns that occur frequently in the data, which can be itemsets, sequences, or other data structures.
- **Use cases**: It is often used to identify frequent itemsets in transactional databases, like finding which items are often purchased together in market basket analysis.

Association Rule Mining

- Association rule mining goes beyond frequent pattern mining by identifying **associations** or relationships between frequent items. It generates rules of the form “If item A, then item B” based on the frequent patterns discovered.
- The goal is to find rules that predict the occurrence of an item based on the occurrence of another item.
- **Use cases**: Commonly used in retail to discover relationships between products and make recommendations based on user behavior.

The Park-Chen-Yu (PCY)

- The Park-Chen-Yu (PCY) algorithm is a well-known method in data mining, particularly used for finding frequent itemsets in large databases, which is a fundamental step in market basket analysis and association rule learning. The algorithm improves upon the classic Apriori algorithm by utilizing a hash-based approach to reduce the number of candidate itemsets that need to be considered.

Overview of the Park-Chen-Yu Algorithm

- The PCY algorithm consists of two main passes over the data:

1. **First Pass**: Count the frequency of individual items and generate candidate pairs (2-itemsets).
2. **Second Pass**: Use a hash table to filter these candidate pairs by counting their occurrences in the transaction database.

Advantages of the Park-Chen-Yu Algorithm

- **Efficiency**: By using a hash table, the algorithm reduces the number of candidate pairs, which minimizes memory usage and processing time compared to the Apriori algorithm.
- **Scalability**: It scales better with larger datasets, making it suitable for big data scenarios.

SON (Savasere, Omiecinski, Navathe) Algorithm

Purpose of SON Algorithm

- The **SON Algorithm** is a variant of the **Apriori Algorithm** used for **frequent itemset mining** on large datasets, especially in distributed systems like **MapReduce**. Instead of processing the entire dataset in one go, it processes it in smaller chunks or partitions to make the mining of frequent patterns more efficient and scalable.

How SON Algorithm Works:

1. **Divide the dataset** into partitions.
2. **Find local frequent itemsets** from each partition using a local support threshold.
3. **Generate candidate frequent itemsets** by combining local frequent itemsets.
4. **Validate candidate itemsets** against the entire dataset to identify globally frequent itemsets.

Key Points of SON Algorithm:

- **Scalability**: The dataset is divided into smaller partitions, making it suitable for large datasets.
- **Efficiency**: It avoids processing the entire dataset at once by using a two-step approach: local frequent itemset mining followed by global validation.
- **Parallelization**: The algorithm works well in distributed environments where different partitions can be processed simultaneously on multiple machines.
- The SON Algorithm allows frequent itemset mining in distributed systems by processing data in smaller chunks and efficiently identifying frequent patterns, even with very large datasets

CURE Algorithm

- The **CURE (Clustering Using Representatives)** algorithm is designed to address the limitations of traditional clustering methods like **k-means** or **hierarchical clustering** when dealing with large datasets, non-spherical clusters, and outliers. It is particularly effective for finding clusters of arbitrary shapes and for handling outliers.

Key Characteristics of the CURE Algorithm:

- **Scalability**: CURE is scalable to large datasets through the use of random sampling and partitioning techniques.
- **Handling Non-Spherical Clusters**: It can cluster datasets that do not form spherical shapes, unlike k-means, which is best suited for spherical clusters.
- **Outlier Handling**: CURE can effectively detect and manage outliers, which is a challenge for many traditional algorithm

Canopy Clustering

- **Canopy Clustering** is a fast, approximate, and scalable clustering algorithm often used as a preprocessing step for more expensive clustering algorithms like k-means or hierarchical clustering. It's particularly useful when dealing with large datasets because it reduces the number of distance computations needed.

Key Concepts:

- **Cheap Distance Metric (Loose Bound)**: Canopy clustering uses a simple, computationally cheap distance metric to determine rough groupings of data points. For example, Manhattan distance or Euclidean distance with some approximations.
- **Two Thresholds (T1 and T2)**: Canopy clustering works with two distance thresholds:
  - **T1** (Loose Threshold): This is a larger distance that determines whether a data point should be considered as part of a "canopy."
  - **T2** (Tight Threshold): This is a smaller distance used to refine the canopy by determining stronger membership within it.
- **Canopy**: A **canopy** is a cluster or group of data points that are close to each other based on the loose threshold (T1). Points within the tight threshold (T2) are considered strongly associated with the canopy, while points only within the loose threshold (T1) are more loosely associated.

Steps of Canopy Clustering:

- **Start with the Dataset**: Assume we have a dataset of points and we need to form clusters.
- **Select a Point Randomly**: Choose a point randomly from the dataset as a "canopy center."
- **Calculate Distance**:
  - Calculate the distance between the selected point and all other points in the dataset using the cheap distance metric.
  - If the distance to a point is less than **T1**, add that point to the current canopy.
  - If the distance is also less than **T2**, mark it for stronger association with the canopy.
- **Remove Points Covered by the Canopy**: Once a canopy is formed, remove the points that are strongly associated (i.e., within distance **T2**) from the dataset to avoid redundant clustering.
- **Repeat Until Dataset is Empty**: Repeat steps 2-4 until no points are left unclustered.
- **Refinement (Optional)**: After forming canopies, apply a more sophisticated clustering algorithm (e.g., k-means) to the points within each canopy to further refine the clustering.

Clustering with MapReduce

- **Clustering with MapReduce** refers to implementing clustering algorithms in a distributed computing framework, such as Hadoop's MapReduce. MapReduce is designed to process large datasets by breaking the problem into two stages: the **Map** and **Reduce** phases, making it well-suited for scalable clustering algorithms like **k-means**.

Clustering with MapReduce: Key Concepts

- **Map Phase**:
  - Each mapper receives a subset of the data and performs a local computation.
  - For clustering, each data point is assigned to its nearest cluster center.
  - The mapper emits key-value pairs where the key is the cluster ID and the value is the data point or the information needed to update the cluster.
- **Reduce Phase**:
  - The reducer aggregates the points assigned to each cluster.
  - It updates the cluster centroid by calculating the new mean of the points in the cluster.
- **Iteration**:
  - The MapReduce job is iteratively run until the cluster centers stabilize (i.e., the centroids no longer change significantly between iterations).

- Example: k-means Clustering with MapReduceLet’s use k-means clustering as an example to explain how it works with MapReduce. In k-means clustering, the goal is to partition a dataset into k clusters, where each point belongs to the cluster with the nearest

	X	Y
P1	1	2
P2	2	1
P3	3	4
P4	5	7
P5	6	8
P6	8	6

- **Step-by-Step Process for Clustering with MapReduce:**
- **Initialize Cluster Centroids**: Assume we are starting with two cluster centroids:
  - Centroid C1: (2, 2)
  - Centroid C2: (7, 7)
- **Map Phase**:
- Each mapper will receive a subset of points from the dataset. The mapper will:
  - Compute the distance of each point to the centroids.
  - Assign the point to the nearest centroid.
  - Emit the key-value pairs, where the key is the cluster ID (C1 or C2), and the value is the point.
- For example, Mapper 1 might process points **P1, P2, and P3** and Mapper 2 processes **P4, P5, and P6**.
  - **Mapper 1**:
    - Distance from P1 (1,2) to C1 (2,2) = 1, Distance to C2 (7,7) = √61 ≈ 7.81 → Assign to C1
    - Distance from P2 (2,1) to C1 (2,2) = 1, Distance to C2 (7,7) = √61 ≈ 7.81 → Assign to C1
    - Distance from P3 (3,4) to C1 (2,2) = √5 ≈ 2.24, Distance to C

Classification Algorithms: 1. Parallel Decision Trees

- A **decision tree** is a classification model that splits the dataset into subsets based on feature values, using tree-like structures. **Parallel Decision Trees** extend this concept by distributing the training process across multiple machines to handle large datasets more efficiently.
- **Example: Building a Parallel Decision Tree for Big Data**
- Imagine you have a large dataset of customer information to predict whether a customer will buy a product (binary classification: "Yes" or "No"). Features could include age, income, and browsing behavior.
- **Steps**:
- **Data Partitioning**: The dataset is partitioned into smaller subsets. Each machine handles a subset of the data.
- **Local Trees**: Each machine builds a decision tree using its partition. For example, one machine may create a decision tree using the "income" feature as the root node.
- **Aggregation**: After local trees are built, the results (trees) are aggregated into a global model. In this step, either a consensus is formed, or models are combined by averaging predictions.
- **Benefit**: By distributing the process across multiple machines, large datasets can be processed more efficiently, reducing training time significantly.

2. Support Vector Machine (SVM) Classifiers

- A **Support Vector Machine (SVM)** classifier is a supervised learning model that finds the hyperplane which best separates data points belonging to different classes. It is highly effective for high-dimensional data and binary classification tasks.
- **Example: SVM for Document Classification**
- Imagine you are building a classifier to categorize documents as "Sports" or "Politics" based on word frequencies.
- **Steps**:
- **Feature Extraction**: Convert the text data into numerical feature vectors (e.g., using term frequency or TF-IDF scores).
- **Training**: The SVM algorithm will find the hyperplane that maximally separates the documents into two categories (sports and politics) by focusing on the most important data points (support vectors).
- **Prediction**: For a new document, the SVM classifier will determine on which side of the hyperplane the document lies, assigning it to either the "Sports" or "Politics" class.
- **Benefit**: SVM works well even with sparse and high-dimensional datasets like text data and performs robustly in many big data applications.

3. K-Nearest Neighbor (KNN) Classifications

- The **K-Nearest Neighbor (KNN)** algorithm is a simple, instance-based learning method. It classifies new data points based on the majority label of the **k** nearest neighbors in the training dataset.
- **Example: KNN for Image Classification**
- Imagine you have a dataset of labeled images of cats and dogs. You want to classify a new image as either a "cat" or "dog."
- **Steps**:
- **Feature Extraction**: Each image is represented by a feature vector (e.g., pixel intensities, edges).
- **Distance Calculation**: For a new image, the KNN algorithm calculates the distance (e.g., Euclidean distance) between the new image and all images in the training set.
- **Voting**: The algorithm selects the **k** nearest neighbors (e.g., k=3). If 2 of the 3 nearest neighbors are labeled "cat," the new image is classified as "cat."
- **Benefit**: KNN is simple to implement and works well when the decision boundary is non-linear. However, it becomes computationally expensive with large datasets, which can be addressed using techniques like **MapReduce** or KD-trees.

4. One Nearest Neighbor (1-NN)

- The **One Nearest Neighbor (1-NN)** algorithm is a special case of KNN where **k=1**. It classifies a new data point based on its nearest neighbor in the training set.
- **Example: 1-NN for Real-Time Traffic Prediction**
- Imagine you have a dataset of traffic conditions with features like time of day, weather, and current traffic volume, and the labels are "light," "moderate," or "heavy" traffic. You want to predict the traffic condition at a new location.
- **Steps**:
- **Feature Representation**: Each traffic data point is represented by its features (time, weather, etc.).
- **Distance Calculation**: For a new data point, the algorithm calculates the distance to every data point in the training set.
- **Prediction**: The label of the closest data point (the one with the smallest distance) is assigned to the new data point.
- **Benefit**: One Nearest Neighbor is simple and effective for real-time predictions, but can be sensitive to noise and outliers. For large datasets, it can be slow, but optimizations like indexing structures (KD-trees) can improve performance.

5. Logistic Regression

- **Logistic regression** is a statistical model used for binary classification. It models the probability of a binary outcome based on one or more input features. The algorithm uses the logistic (sigmoid) function to map predictions to probabilities.
- **Example: Logistic Regression for Customer Churn Prediction**
- Imagine you have customer data (e.g., usage time, number of support tickets, product type) and you want to predict whether a customer will "churn" (stop using the service).
- **Steps**:
- **Model the Probability**: Logistic regression will calculate the probability that a customer will churn (1 = churn, 0 = no churn) based on input features.
- **Sigmoid Function**: The algorithm uses the sigmoid function to convert the linear combination of the input features into a probability (a value between 0 and 1)
- **Threshold**: If the probability is greater than a threshold (e.g., 0.5), predict "churn," otherwise predict "no churn."
- **Benefit**: Logistic regression is interpretable (coefficients show feature impact) and efficient, making it suitable for large-scale classification tasks in big data settings.

Simple Linear Regression

- Simple linear regression is a statistical method used to model the relationship between two variables by fitting a linear equation to the observed data. It aims to predict the value of a dependent variable (Y) based on the value of one independent variable (X).
- **Applicability**:
- Best used when there is a clear linear relationship between the independent variable and the dependent variable.
- Commonly applied in fields like economics, biology, and social sciences for predicting outcomes based on a single factor.
- **Example**: Suppose we want to predict a student's exam score based on the number of hours they studied. We collect data on hours studied and corresponding exam scores:

Hours Studied (X)	Exam Score (Y)
1	50
2	60
3	70
4	80
5	90

The simple linear regression equation can be represented as:

$Y=a+bX$

Where:

- Y = predicted exam score
- a = y-intercept (score when hours studied = 0)
- b = slope (change in score per additional hour studied)

Assuming the regression analysis gives us $a=50$ and $b=10$, the equation becomes:

$Y=50+10X$

This model allows us to predict that if a student studies for 3 hours, their predicted score would be:

$Y=50+10(3)=80$

Complexity:

- Simple linear regression is relatively straightforward to implement and interpret.
- The model complexity is low, involving only two parameters (slope and intercept).

Multiple Linear Regression

- Multiple linear regression extends simple linear regression by modeling the relationship between a dependent variable and multiple independent variables. It aims to predict the dependent variable using multiple predictors.
- **Applicability**:
- Useful when a dependent variable is influenced by multiple factors.
- Common in fields like finance, healthcare, and marketing for predicting outcomes based on several variables.
- **Example**: Suppose we want to predict a house's selling price based on various features: square footage, number of bedrooms, and age of the house. We might collect the following data:

Square Footage (X1)	Bedrooms (X2)	Age (X3)	Price (Y)
1500	3	10	$300,000
2000	4	5	$400,000
2500	4	2	$500,000
1800	3	15	$275,000
2200	4	8	$450,000

The multiple linear regression equation can be expressed as:

$Y=a+b1X1+b2X2+b3X3$

Where:

- Y = predicted price
- a = y-intercept
- $b1,b2,b3$ = coefficients representing the impact of each independent variable.

Assuming regression analysis provides $a=50,000$, $b1=100$(per square foot), $b2=20,000$ (per bedroom), and $b3=-2,000$ (per year of age), the equation becomes:

$Y=50,000+100X1+20,000X2-2,000X3$

If a house has 2000 square feet, 4 bedrooms, and is 5 years old, the predicted price would be:

$Y=50,000+100(2000)+20,000(4)-2,000(5)$

$Y=50,000+200,000+80,000-10,000=320,000$

Complexity:

- Multiple linear regression is more complex than simple linear regression as it involves more parameters and potential interactions between variables.
- Requires more data to estimate multiple coefficients accurately and can be prone to overfitting if too many variables are included without justification.

Visual data analysis

- Visual data analysis techniques aim to enhance the understanding of complex datasets by transforming abstract data into visual representations. Here are the main goals of these techniques and how they facilitate better comprehension:

Main Goals of Visual Data Analysis

1. **Data Exploration**:
    1. Allow users to interactively explore data and discover patterns, trends, and anomalies.
    2. Tools like scatter plots, heat maps, and parallel coordinates enable users to quickly identify relationships and distributions, leading to deeper insights.
2. **Pattern Recognition**:
    1. Help users recognize underlying patterns and trends that may not be apparent in raw data.
    2. Visualizations such as line graphs and bar charts highlight changes over time or categorical comparisons, making it easier to spot trends or recurring behaviors.
3. **Data Communication**:
    1. Convey complex information clearly and effectively to a diverse audience.
    2. Well-designed visualizations (e.g., dashboards, infographics) use visual hierarchy and color coding to emphasize key points, making data accessible to non-experts.
4. **Decision Support**:
    3. Assist in informed decision-making based on data insights.
    4. Interactive visualizations allow users to simulate scenarios and assess outcomes, helping stakeholders make data-driven decisions confidently.

5. **Outlier Detection**:
    5. Identify data points that deviate significantly from expected patterns.
    6. Box plots and scatter plots make it easy to spot outliers, allowing for further investigation and quality control.

6. **Contextualization**:
    7. Provide context to the data by integrating additional information.
    8. Visualizations can incorporate geographical maps, timelines, or annotations to enrich the data story, helping users understand the context behind the numbers.

Important Topics

- main focus of frequent pattern mining, and its important in data analysis
- fundamental principle behind clustering algorithms and their purpose in data analysis.
- the applications of Canopy Algorithm
- the main goals of visual data analysis techniques and how they facilitate better understanding of complex datasets.
- the advantages of PCY algorithm?
- How large data handled in main memory? Enlist the algorithm used to handle large datasets in main memory?
- all three pass of multistage algorithm

- Given a large dataset and memory constraints, outline the steps of the basic Park, Chen, and Yu algorithm for frequent pattern mining.
- List down two algorithms used for frequent pattern mining? Describe their key characteristics in detail.
- Analyse the strengths and limitations of the CURE algorithm for clustering in comparison to traditional centroid-based clustering algorithms.
- Compare the Canopy clustering approach to hierarchical agglomerative clustering. What are the computational and memory trade-offs between these methods?
- Illustrate how the SON partition-based algorithm helps to perform frequent item set mining for large datasets. How does this algorithm avoid False negatives?
- What is the primary function of visualization techniques in data analysis, and how do they aid in extracting insights from data?
- Analyse the impact of varying the number of principal components retained after PCA on the quality of data representation and the explanatory power of the reduced dimensions.

- Analyse clustering with MapReduce.
- Illustrate how the PCY algorithm is used in big data analytics.
- "Apply PCY algorithm on the following transactions to find candidate set. use buckets and the concept of mapreduce to solve the problem. Threshhold:2, Hash Function: (i*j) mod 10T1:{1,2,3} T2:{2,3,4} T3:{3,4,5} T4:{4,5,6} T5:{1,3,5} T6:{2,4,6} T7:{1,3,4} T8:{2,4,5} T9:{3,5,6} T10:{1,2,4} T11:{2,3,5} T12:{3,4,6}"
- Apply different classification algorithms to a sample dataset. Describe how you would implement algorithms such as Decision Trees, Support Vector Machines (SVM), K-Nearest Neighbors (K-NN), and Logistic Regression. Explain the steps involved in each algorithm and how you would evaluate their performance using metrics like accuracy, precision, and recall.
- Compare and contrast the strengths and limitations of simple linear regression and multiple linear regression models in terms of their applicability and complexity.
- Given a dataset with a high number of features, outline the steps you would take to perform dimension reduction using PCA. What considerations would guide your choices?
- How does the SON (Savasere, Omiecinski, and Navathe) algorithm leverage MapReduce to mine frequent patterns? What advantage does MapReduce bring to this process?